

Chương 5: Máy học Véc-tơ Hỗ trợ (Support Vector Machines)

Giảng viên

Đại học Đông Á

August 11, 2026

Mục tiêu của Chương

- Hiểu khái niệm cốt lõi của **Support Vector Machines (SVM)**.
- Phân biệt giữa **Biên cứng (Hard Margin)** và **Biên mềm (Soft Margin)** trong phân loại SVM tuyến tính.
- Xử lý dữ liệu phi tuyến bằng kỹ thuật bổ sung đặc trưng hoặc **Kernel trick**.
- Áp dụng **Kernel Đa thức (Polynomial)** và **Kernel Gaussian RBF**.
- Khám phá cách sử dụng SVM cho bài toán **Hồi quy (Regression)**.
- Đi sâu vào **toán học phía sau SVM**: Bài toán tối ưu, Hàm mục tiêu và Đối ngẫu.

Mục lục

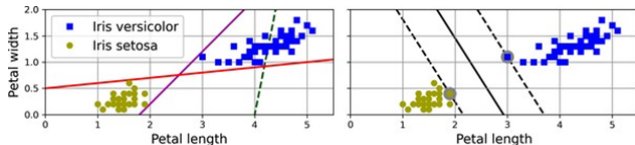
1. Phân loại SVM Tuyến tính
2. Phân loại SVM Phi tuyến
3. Lớp SVM & Hồi quy SVM
4. Toán học: Bên trong Thuật toán SVM
5. Tổng kết chương

1.1 Ý tưởng cơ bản của SVM

- SVM là một trong những mô hình phổ biến và mạnh mẽ nhất trong Học máy truyền thống.
- SVM có thể giải quyết các bài toán phân loại tuyến tính, phi tuyến, và cả hồi quy.
- Tốt nhất cho các tập dữ liệu có kích thước **vừa và nhỏ** nhưng cực kỳ phức tạp.
- Ý tưởng chính: Tìm ra một **đường thẳng (hoặc siêu phẳng)** chia tách các lớp dữ liệu sao cho **khoảng cách** từ đường thẳng tới các điểm gần nhất của mỗi lớp là lớn nhất.

1.2 Mô hình Phân loại Biên Lớn (Large Margin Classification)

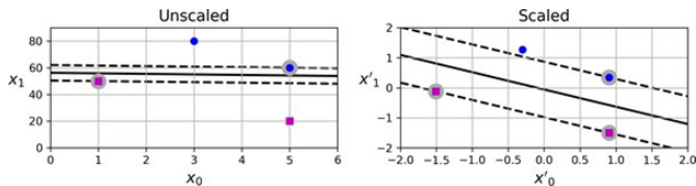
- Đường đứt nét ở giữa chia hai lớp một cách rõ ràng.
- Khoảng cách từ đường nét liền đến điểm dữ liệu gần nhất gọi là **Margin (Biên)**.
- SVM cố gắng làm cho lề này rộng nhất có thể.
- Các điểm nằm ngay trên rìa của lề được gọi là các **Véc-tơ Hỗ trợ (Support Vectors)**.



Hình 5-1: Ý tưởng Large Margin Classification

1.3 Tính nhạy cảm với Tỷ lệ Đặc trưng (Feature Scaling)

- SVM rất nhạy cảm với tỷ lệ của các đặc trưng (thang đo).
- Nếu các đặc trưng có tỷ lệ quá chênh lệch (VD: x_1 từ 0-1000, x_2 từ 0-1), ranh giới quyết định sẽ gần như song song với trục lớn hơn.
- Do đó, luôn cần thực hiện **StandardScaler** trước khi đưa dữ liệu vào SVM.

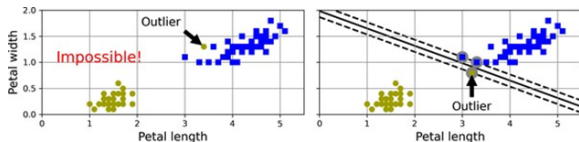


Hình 5-2: Sự khác biệt của lề khi chưa scale và đã scale

1.4 Phân loại Biên Cứng (Hard Margin Classification)

- Cố gắng chia tách tất cả các điểm một cách **hoàn hảo tuyệt đối**. Không điểm nào được phép nằm trong vùng lề hoặc nằm sai phía ranh giới.
- **Vấn đề 1:** Chỉ hoạt động được khi dữ liệu phân tách tuyến tính (Linearly Separable).
- **Vấn đề 2:** Rất nhạy cảm với Điểm bất thường (Outliers). Một nhiễu nhỏ cũng có thể làm biến dạng ranh giới.

1.5 Tác động của Điểm bất thường (Outliers)



Hình 5-3: Biên Cứng bị bóp méo hoặc thất bại khi có outlier

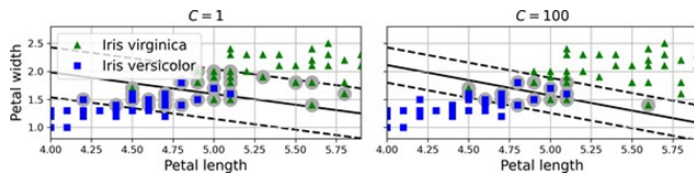
- Hình bên trái: Một điểm xanh rơi vào vùng lớp đỏ khiến thuật toán không thể vẽ được đường thẳng chia cách hoàn hảo.
- Hình bên phải: Một outlier làm lệch bị co hẹp lại một cách cực đoan, mô hình mất tính tổng quát.

1.6 Phân loại Biên Mềm (Soft Margin Classification)

- Để tránh những vấn đề của Biên cứng, ta sử dụng một mô hình linh hoạt hơn.
- Mục tiêu mới: Tìm sự cân bằng giữa việc **giữ cho lề rộng nhất có thể** và **hạn chế tối đa các điểm vi phạm lề** (Margin Violations).
- Đây gọi là Phân loại Biên mềm (Soft Margin Classification).

1.7 Siêu tham số C trong Scikit-Learn

- Tham số 'C' điều khiển sự cân bằng trong Biên mềm.
- **C nhỏ:** Lề rộng hơn, nhưng cho phép nhiều điểm vi phạm lề (Tăng bias, Giảm variance). Dễ bị Underfitting.
- **C lớn:** Lề hẹp lại để hạn chế vi phạm lề (Giảm bias, Tăng variance). Dễ bị Overfitting.



Hình 5-4: So sánh khi C thấp (trái) và C cao (phải)

1.8 Triển khai LinearSVM bằng Python

```
import numpy as np
from sklearn import datasets
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC

# Tai du lieu Iris (Hoa diem vi)
iris = datasets.load_iris()
X = iris["data"][:, (2, 3)] # Chieu dai, chieu rong canh hoa
y = (iris["target"] == 2).astype(np.float64) # Iris virginica

# Xay dung Pipeline: chuan hoa + huan luyen
svm_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("linear_svc", LinearSVC(C=1, loss="hinge", random_state=42))
])

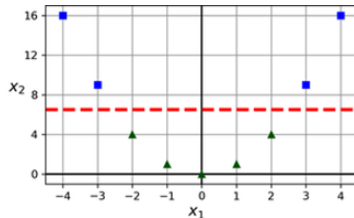
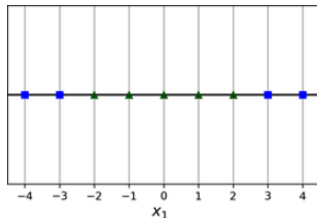
svm_clf.fit(X, y)
print("Du doan:", svm_clf.predict([[5.5, 1.7]])) # Output: [1.]
```

2.1 Giới thiệu SVM Phi tuyến

- Rất nhiều bộ dữ liệu thực tế không thể phân tách bằng một đường thẳng.
- Một phương pháp đơn giản để xử lý tập dữ liệu phi tuyến là **thêm nhiều đặc trưng (Features) mới**, chẳng hạn như đặc trưng đa thức (Polynomial).
- Dữ liệu gốc ở 1D có thể không tách được, nhưng khi chiếu lên không gian 2D, ta có thể dùng đường thẳng để tách!

2.2 Minh họa Thêm đặc trưng đa thức

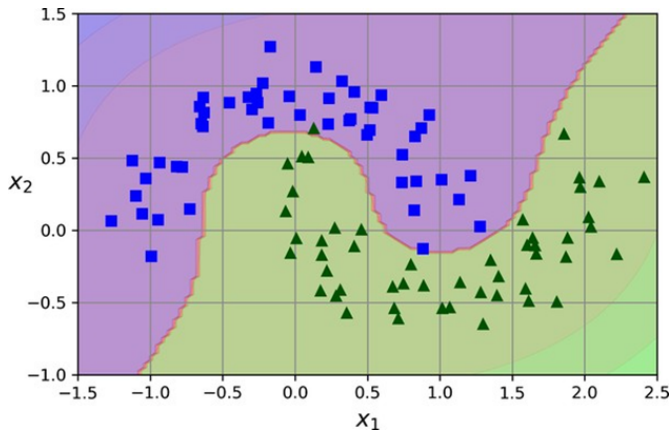
- Hình bên trái: Các điểm đỏ và xanh trên một trục 1D (x_1). Rõ ràng không có 1 điểm cắt nào phân chia hoàn hảo.
- Khi ta thêm đặc trưng mới $x_2 = (x_1)^2$.
- Hình bên phải: Trong không gian 2D (x_1, x_2), dữ liệu đã trở nên **phân tách tuyến tính**.



Hình 5-5: Thêm đặc trưng $x_2 = x_1^2$ giúp phân tách tuyến tính

2.3 Phân tách Dataset "Moons" (Hai nửa vành trăng)

- Bộ dữ liệu *Make Moons* gồm 2 nửa vành khuyên lồng vào nhau.
- Sử dụng 'PolynomialFeatures' kết hợp 'StandardScaler' và 'LinearSVC'.
- Đường cong ranh giới quyết định rất linh hoạt và phân loại chính xác các điểm.



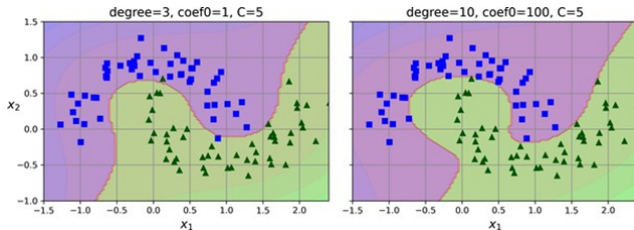
Hình 5-6: Ranh giới quyết định của Linear SVM với Polynomial Features

2.4 Kernel Trick (Thủ thuật Kernel)

- Việc tạo ra đặc trưng đa thức có bậc cao (Degree = 10, 20) sẽ tạo ra vô số các đặc trưng mới, làm chậm mô hình (Khủng hoảng chiều không gian - Curse of Dimensionality).
- May mắn thay, với SVM ta có một phép màu toán học gọi là **Kernel trick**.
- **Kernel Trick** cho phép SVM tính toán ra kết quả phân loại giống y hệt như việc ta thêm hàng tỉ đặc trưng đa thức, nhưng thực tế **nó không hề thêm một đặc trưng nào**. Do đó, không làm chậm thuật toán!

2.5 Kernel Đa thức (Polynomial Kernel)

- 'SVC(kernel="poly", degree=3)'
- Tham số 'coef0': Kiểm soát mức độ ảnh hưởng của đa thức bậc cao so với đa thức bậc thấp.
- Hình ảnh cho thấy sự khác biệt giữa mô hình bậc 3 (trái) và bậc 10 (phải).



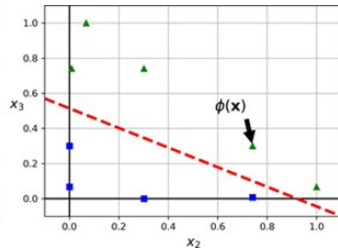
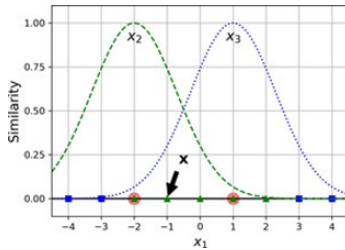
Hình 5-7: Polynomial Kernel với bậc 3 và 10

2.6 Đặc trưng tương tự (Similarity Features)

- Một kỹ thuật khác: Bổ sung các đặc trưng sử dụng **Hàm tương tự (Similarity Function)**, đo lường mỗi mẫu giống các Mốc (Landmarks) đến mức nào.
- Ví dụ: Gaussian Radial Basis Function (RBF).
- Công thức: $\phi_{\gamma}(x, \ell) = \exp(-\gamma ||x - \ell||^2)$
- Giá trị này dạng hình chuông, thay đổi từ 0 (rất xa) đến 1 (ngay tại mốc).

2.7 Minh họa Hàm Gaussian RBF

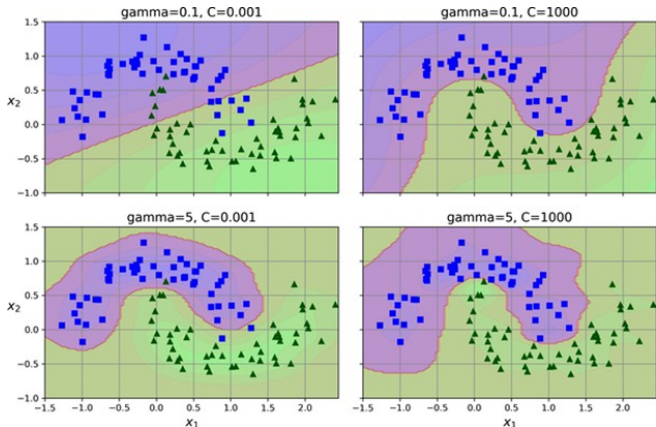
- Đặt mốc tại $x_1 = -2$ và $x_1 = 1$.
- Chuyển điểm $x = -1$ thành 2 chiều mới: khoảng cách tới mốc 1 và khoảng cách tới mốc 2.
- Giờ đây tập dữ liệu đã có thể phân tách tuyến tính bằng 1 đường thẳng.



Hình 5-8: Sử dụng Similarity Features phân tách dữ liệu 1D

2.8 Gaussian RBF Kernel

- Giống đa thức, ta dùng **Kernel RBF** để hưởng lợi ích của vô vàn các Mốc (Landmarks) mà không phải tính toán chi phí cao.
- 'SVC(kernel="rbf", gamma=5, C=0.001)'
- Tham số 'gamma' (γ): Định hình độ hẹp/rộng của quả chuông.



Hình 5-9: Hiệu ứng của γ và C lên SVM Kernel RBF

2.9 Điều chỉnh siêu tham số Gamma và C

- **Gamma (γ) lớn:** Quả chuông nhọn hẹp. Ranh giới uốn lượn sát theo từng điểm dữ liệu (Overfitting).
- **Gamma (γ) nhỏ:** Quả chuông rộng, mượt. Ranh giới mượt mà hơn (Underfitting).
- Lời khuyên: Nếu mô hình bị Overfitting, hãy giảm γ . Ngược lại, hãy tăng γ . (Tương tự như C).
- Rule of thumb: Khi nghi ngờ, hãy luôn thử **Linear Kernel** trước, sau đó tới **Gaussian RBF Kernel**.

3.1 Các Lớp của Scikit-Learn cho SVM

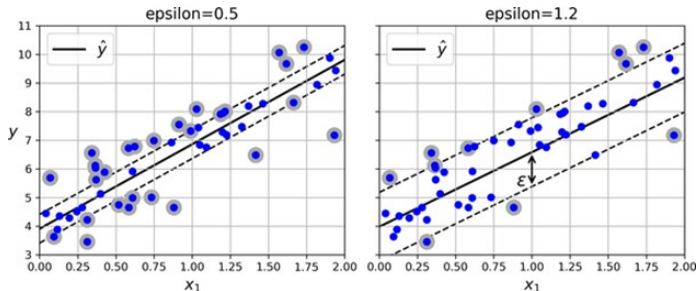
- 'LinearSVC': Dùng thư viện LIBLINEAR, tối ưu cho tập dữ liệu cực lớn, phức tạp thời gian $\mathcal{O}(m \times n)$. (Chỉ chạy tuyến tính).
- 'SVC': Dùng thư viện LIBSVM, hỗ trợ Kernel trick. Phức tạp thời gian $\mathcal{O}(m^2 \times n)$ đến $\mathcal{O}(m^3 \times n)$. (Quá chậm nếu số lượng mẫu $> 100,000$).
- 'SGDClassifier(loss="hinge")': Huấn luyện Gradient Descent ngẫu nhiên, hữu ích cho tập dữ liệu khổng lồ (Out-of-core learning).

3.2 Ý tưởng Hồi quy bằng SVM

- Thuật toán SVM cực kỳ đa năng. Nó không chỉ phân loại tốt mà còn hồi quy (Dự đoán giá trị số liên tục) xuất sắc!
- Trong **Phân loại SVM**, mục tiêu là để dải lề rộng nhất nhưng KHÔNG chứa bất cứ điểm vi phạm nào bên trong dải lề.
- Ngược lại, trong **Hồi quy SVM**, mục tiêu là đưa CÀNG NHIỀU ĐIỂM VÀO BÊN TRONG dải lề càng tốt, trong khi hạn chế số điểm rơi ra ngoài dải lề.
- Độ rộng lề được kiểm soát bởi siêu tham số Epsilon (ϵ).

3.3 Minh họa SVM Regression Tuyến tính

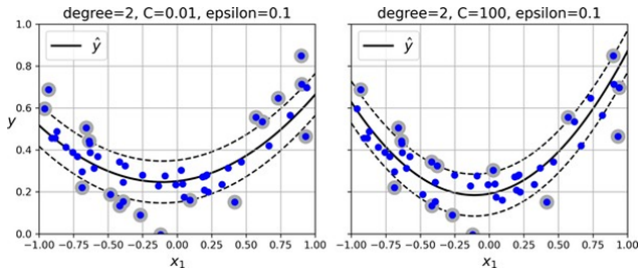
- Hình bên minh họa 'LinearSVR'.
- Việc thêm nhiều điểm dữ liệu nằm bên trong biên không làm ảnh hưởng đến dự đoán của mô hình.
- Mô hình gọi là ϵ -insensitive.
- ϵ lớn (trái) lề rộng. ϵ nhỏ (phải) lề hẹp.



Hình 5-10: Hồi quy LinearSVR với các giá trị ϵ khác nhau

3.4 Hồi quy SVM Phi tuyến (Nonlinear SVR)

- Hồi quy SVM cũng hỗ trợ Kernel trick thông qua lớp 'SVR(kernel="poly")'.
- Trong hình là dữ liệu theo dạng hàm bậc 2. Mô hình có thể bám rất sát xu hướng với đường cong mịn màng.
- C nhỏ (trái): Mô hình tự do, linh hoạt.
- C lớn (phải): Ít sai số, sát dữ liệu hơn (ít regularization).



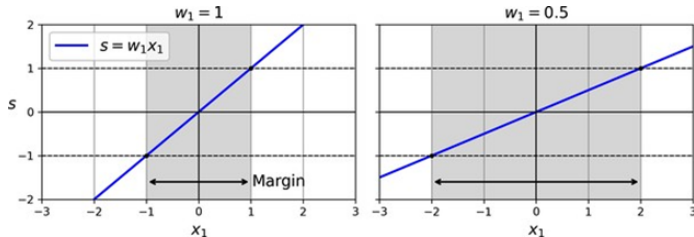
Hình 5-11: Hồi quy SVM phi tuyến bằng Kernel Đa thức

4.1 Hàm Quyết định (Decision Function)

- SVM tuyến tính dự đoán bằng cách tính hàm quyết định (Decision Function):
- Hàm số: $f(\mathbf{x}) = \mathbf{w}^T \cdot \mathbf{x} + b = w_1x_1 + \dots + w_nx_n + b$
- Nếu kết quả dương, dự đoán lớp 1 (Tích cực). Nếu âm, dự đoán lớp 0 (Tiêu cực).
- $\hat{y} = 1$ if $\mathbf{w}^T \mathbf{x} + b \geq 0$, else 0.

4.2 Đồ thị Không gian Hàm quyết định

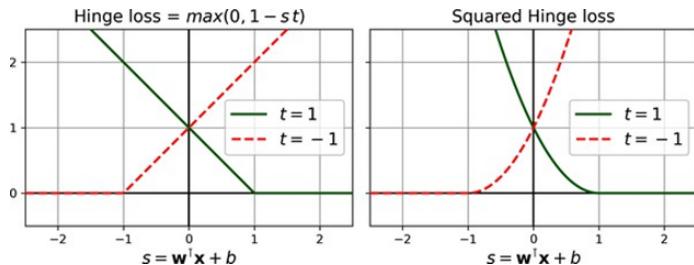
- Không gian 3 chiều: Trục Z là giá trị của hàm quyết định.
- Mặt phẳng cắt tại $Z = 0$ tạo thành ranh giới quyết định (Đường liền đen).
- Đường lề đứt nét tương ứng với mặt cắt $Z = 1$ và $Z = -1$.



Hình 5-12: Decision function cho mô hình Iris Dataset 2D

4.3 Mối liên hệ giữa Vector Trọng số ($\mathbf{w}$) và Biên

- Kích thước của véc-tơ trọng số $\mathbf{w}$ xác định độ dốc của mặt phẳng hàm.
- Nếu chia đôi độ dốc $\mathbf{w}$, độ rộng của lề (Margin) sẽ nhân đôi!
- Để có Margin cực đại, ta phải **tối thiểu hóa độ lớn của $\mathbf{w}$** , tức là $\|\mathbf{w}\|$.



Hình 5-13: Trọng số $\mathbf{w}$ càng nhỏ, Lề margin càng lớn

4.4 Hàm Mục Tiêu của Phân loại Biên Cứng

- Ta muốn tối thiểu hóa $\|\mathbf{w}\|$ (để lề rộng nhất).
- Điều kiện ràng buộc: Tất cả mẫu lớp 1 phải có $f(\mathbf{x}) \geq 1$, và lớp 0 phải có $f(\mathbf{x}) \leq -1$.
- Đặt $t^{(i)} = 1$ nếu $y^{(i)} = 1$ và $t^{(i)} = -1$ nếu $y^{(i)} = 0$. Ràng buộc gộp lại thành:
 $t^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1$ với mọi mẫu.
- Bài toán tối ưu hóa có ràng buộc:
- Cực tiểu hóa: $\frac{1}{2} \mathbf{w}^T \mathbf{w}$
- Sao cho: $t^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1 \quad \forall i$

4.5 Hàm Mục Tiêu của Phân loại Biên Mềm

- Trong Biên mềm, ta chấp nhận lỗi. Ta định nghĩa biến Slack $\zeta^{(i)} \geq 0$.
- $\zeta^{(i)}$ đo lường mức độ mẫu thứ i được phép vi phạm lề.
- Tối thiểu hóa 2 mục tiêu mâu thuẫn nhau: làm cho $\mathbf{w}$ nhỏ nhất và làm cho tổng ζ nhỏ nhất.
- Bài toán tối ưu: Cực tiểu hóa $\frac{1}{2}\mathbf{w}^T\mathbf{w} + C\sum_{i=1}^m \zeta^{(i)}$
- Siêu tham số C xác định sự đánh đổi giữa hai mục tiêu này.

4.6 Trình giải Bài toán (Solvers)

- Đây là các bài toán **Quy hoạch Toàn phương (Quadratic Programming - QP)** có lời giải tối ưu toàn cục.
- Có rất nhiều solver có sẵn để giải: SMO, LIBLINEAR, LIBSVM.
- Có một dạng khác: Dùng Gradient Descent thông thường để cực tiểu hóa Hinge Loss. Đó chính là 'SGDClassifier'.

4.7 Hinge Loss

- Hàm mất mát Hinge: $\max(0, 1 - t)$
- Nó bằng 0 nếu $t \geq 1$ (điểm đã qua khỏi dải lề và nằm đúng phần).
- Nó tăng tuyến tính khi $t < 1$. Điều này tạo ra hình dạng cái bản lề.
- Hàm này không có đạo hàm tại $t = 1$, nhưng vẫn dùng Subgradient để tối ưu bằng SGD được.

4.8 Bài toán Đối ngẫu (Dual Problem)

- Bất kỳ bài toán Tối ưu hóa có ràng buộc (Primal Problem) nào cũng có một bài toán song sinh đi kèm gọi là bài toán Đối ngẫu (Dual Problem).
- Thường thì giải Dual dễ hơn Primal trong SVM khi số lượng quan sát m bé hơn số lượng đặc trưng n .
- Quan trọng hơn: Nhờ dạng Đối ngẫu mà phép màu **Kernel trick** mới có thể thực hiện được, điều mà dạng Primal không làm được.

4.9 Đẳng thức Kernel (Mercer's Theorem)

- Thay vì ánh xạ điểm x qua $\phi(x)$ (hàm biến đổi đa thức khổng lồ), ta chỉ cần tìm ra một hàm $K(\mathbf{a}, \mathbf{b})$.
- Hàm Kernel K tính ra **tích vô hướng** của $\phi(\mathbf{a})^T \phi(\mathbf{b})$ cực kì nhanh gọn.
- Các Kernel phổ biến:
 - Tuyến tính: $K(\mathbf{a}, \mathbf{b}) = \mathbf{a}^T \mathbf{b}$
 - Đa thức: $K(\mathbf{a}, \mathbf{b}) = (\gamma \mathbf{a}^T \mathbf{b} + r)^d$
 - Gaussian RBF: $K(\mathbf{a}, \mathbf{b}) = \exp(-\gamma \|\mathbf{a} - \mathbf{b}\|^2)$
 - Sigmoid: $K(\mathbf{a}, \mathbf{b}) = \tanh(\gamma \mathbf{a}^T \mathbf{b} + r)$

5.1 Ôn tập: Kiến trúc của SVM

- SVM là một mô hình phân loại dựa trên khoảng cách (Lề phân cách).
- Có hai biến thể là Tuyến tính (Dùng Hinge Loss, Solver cực nhanh) và Phi tuyến tính (Dùng Kernel).
- Lề cứng nghiêm ngặt tuyệt đối, lề mềm dùng tham số C để cân bằng lỗi.
- Các bài toán được mô hình hóa chặt chẽ dựa trên Quy hoạch Toàn phương.

5.2 Ứng dụng phổ biến của SVM

- Phân loại văn bản và phân tích cú pháp (Text/Categorization)
- Nhận dạng hình ảnh (Image recognition, MNIST)
- Phân loại gen tin sinh học (Bioinformatics, ung thư, protein)
- Hồi quy dự đoán tài chính.

5.3 Ưu điểm của SVM

- Hiệu quả với dữ liệu có chiều cao (Số lượng Đặc trưng lớn).
- Vẫn tối ưu cực tốt khi Không gian chiều dữ liệu lớn hơn cả số mẫu ($n > m$).
- Tiết kiệm bộ nhớ vì hàm ra quyết định chỉ phụ thuộc một nhóm các Support Vectors.
- Có nền tảng lý thuyết tối ưu lồi (Convex Optimization) bảo đảm hội tụ tới Tối ưu Toàn cục (Global Optimum), không bị dính vào tối ưu cục bộ như Neural Networks.

5.4 Nhược điểm của SVM

- Thời gian huấn luyện ($O(n^2)$ đến $O(n^3)$) rất chậm nếu bộ dữ liệu có trên 100,000 dòng.
- Không thể cho ra Xác suất (Probability) một cách tự nhiên như Logistic Regression (Phải dùng Platt Scaling tốn kém).
- Cực kỳ nhạy cảm và bắt buộc phải có bước chuẩn hóa (Scale) dữ liệu.
- Không diễn giải dễ dàng (Black-box model).

5.5 Bài tập Thực hành: Phân loại Ảnh Chữ Số

- Đề bài: Xây dựng bộ phân loại SVM trên bộ dữ liệu MNIST.
- MNIST gồm các ảnh xám 28x28 pixel của các chữ số viết tay từ 0-9.
- Vì SVM mặc định là nhị phân (Binary), nó sẽ sử dụng chiến lược One-versus-Rest (OvR) hoặc One-versus-One (OvO) tự động bên dưới Scikit-Learn.

5.6 Code Gợi ý Huấn luyện MNIST bằng SVM

```
from sklearn.datasets import fetch_openml
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# 1. Tải dữ liệu và Scale (0-255 -> 0-1)
mnist = fetch_openml('mnist_784', version=1)
X, y = mnist["data"], mnist["target"]
X_train, X_test, y_train, y_test = X[:60000]/255., X[60000:]/255., y[:60000], y[60000:]

# 2. Xây dựng mô hình SVM với RBF Kernel (Mặc định)
svm_clf = SVC(gamma="scale")
svm_clf.fit(X_train[:10000], y_train[:10000]) # Huấn luyện tập nhỏ cho nhanh

# 3. Kiểm thử
y_pred = svm_clf.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

5.7 Lựa chọn Mô hình phù hợp

- Đừng lao vào Deep Learning ngay khi làm dự án thực tế.
- Luôn thử Logistic Regression hoặc Random Forest trước làm Baseline.
- Sau đó thử SVM. Dù tốc độ đào tạo chậm, nhưng khả năng phân loại của SVM thường vượt trội với tập dữ liệu tabular (bảng).

HỎI & ĐÁP